

Symbiosis Institute of Technology (SIT), Pune

FixItAI: A Context-Aware Technical Support Retrieval Workspace

Project Report

Submitted to

Dr. Payal Kadam

Group Members:

Swarali Bhalerao 23070122216

Tisha Malkani 23070122224

Department of Computer Science and Engineering

Abstract

Technical support questions are often described as symptoms rather than exact error messages. Keyword search can miss relevant solutions when wording differs, while generative systems may provide advice unsupported by their sources. This project develops FixItAI, a local technical-support workspace that retrieves troubleshooting passages and displays their sources. The system combines a React interface, FastAPI service, lexical retrieval using TF–IDF and BM25, and a compact BERT bidirectional encoder initialized randomly with no pretrained weights. The supplied Stack Overflow archive contains 1,264,216 question rows, 2,014,516 answer rows, and 3,750,994 tag rows. The BERT run samples 200,000 questions; 173,857 usable question–answer pairs were selected using the highest-scored associated answer as a weak relevance label and divided into training and validation sets. A tokenizer was learned from the training text, and the BERT encoder produces normalized 128-dimensional representations optimized with symmetric in-batch contrastive loss on CUDA. The BERT experiment completed four CUDA epochs. Epoch 4 recorded Recall@1 6.30%, Recall@5 19.30%, and MRR 0.1313. The best recorded checkpoint by MRR was epoch 3 (MRR 0.1313). For context, a separate earlier scratch Transformer run completed four epochs and obtained MRR 0.0311, Recall@1 0.72%, and Recall@5 4.76%; those are not BERT results. The weak vote-derived labels do not establish technical correctness. Future work includes completing the BERT run and index, controlled lexical baselines, expert judgments, freshness metadata, and broader support documentation.

Keywords: Technical question answering; information retrieval; Transformer; contrastive learning; Stack Overflow; hybrid search; CUDA.

Contents

1. Introduction
2. Literature Review
3. Proposed Methodology
4. Experimental Setup
5. Results and Discussion
6. Comparative Analysis
7. Limitations
8. Future Scope
9. Conclusion
10. CO4 and CO5 Course Coverage
11. References

1. Introduction

Software users often describe a problem through a symptom, an unsuccessful workflow, or a short error message. Different users may describe the same underlying issue using different terminology, application versions, operating systems, and levels of detail. Developers, everyday computer users, and support teams all spend time locating and validating relevant guidance. Community question-and-answer archives contain practical troubleshooting knowledge, but their size and uneven post quality make manual search difficult [1], [2].

Term-based search is efficient and interpretable, but exact lexical overlap is weak when a question and its solution use different vocabulary. Dense retrieval can represent semantic similarity, while lexical signals preserve exact error strings, APIs, version numbers, and identifiers. Benchmark studies show that neither approach dominates every domain, motivating hybrid retrieval and domain-specific evaluation [3], [4].

Generative answer systems can produce fluent responses, but their reliability depends on whether retrieved evidence supports each claim [5]. FixItAI therefore returns ranked passages with source attribution rather than generating new instructions. This design makes evidence traceable and limits unsupported synthesis, but the system cannot answer well when relevant or current material is absent from its index.

The motivation is to build an end-to-end local support application and to evaluate a Transformer trained from random initialization on the available CUDA GPU without pretrained weights. The research gap is practical: many strong embedding systems depend on large pretrained models and extensive infrastructure, whereas a compact, locally trained, source-linked retriever is easier to inspect and reproduce on a student workstation. This is an engineering investigation, not a claim that the model is state of the art.

The objectives are to audit the supplied archive, form question–answer pairs, learn a tokenizer and Transformer from scratch, measure retrieval ranking, and expose search through a usable web interface. Major contributions are:

- A local React/FastAPI workspace for technical search with source-linked passages.
- A compact BERT bidirectional encoder initialized from random weights and trained on CUDA for question–answer retrieval (four-epoch CUDA training and evaluation completed).
- A hybrid ranking pipeline combining TF–IDF, BM25, semantic similarity, and technology/error cues.
- An explicit evaluation protocol and discussion of vote-label, sample-size, and GPU-memory limitations.

2. Literature Review

Recent software-support research targets question matching and answer recommendation. TopicAns combines topic-aware text representations with a pretrained encoder to reduce noise in long technical posts and reports gains in Precision@1 on four datasets; its extra topic model and pretrained model increase system complexity [2]. Que2Code retrieves semantically equivalent Stack Overflow questions before recommending code snippets, and includes automatic and human evaluation, but focuses on code fragments rather than general support prose [1]. Repo4QA links programming questions to GitHub repositories using dense retrieval; this addresses repository discovery, not direct answer-passage retrieval [6]. [7] compare representations for Stack Overflow tasks such as tag/API recommendation, showing that embeddings are task-dependent, while ProCQA adds community programming QA data and

contrastive text/code alignment [8]. These studies establish the value of technical-community data but do not remove the need for a small local support tool with explicit evidence links.

Dense retrieval papers provide the modeling basis. DPR uses separate question and passage encodings trained to rank relevant passages [9]. SimCSE shows how contrastive learning can shape sentence embeddings [10], and Contriever demonstrates unsupervised contrastive retrieval across multiple domains [11]. E5 learns from weakly supervised text pairs and reports broad transfer across retrieval tasks [12]. ColBERTv2's token-level late interaction improves fine-grained matching and reduces index size, but still carries a larger multi-vector storage and computation burden than a single-vector dual encoder [13]. Sentence-T5 also reports strong sentence-transfer performance, but uses pretrained text-to-text encoders [14]. FixItAI adopts a smaller single-vector architecture and accepts lower expected capacity in exchange for scratch training and local operation.

Benchmark research warns that embedding quality is task- and domain-dependent. BEIR found BM25 to be a robust zero-shot baseline while neural methods varied across datasets (Thakur et al., 2021). MTEB evaluates multiple embedding tasks and finds no universal model winner (Muennighoff et al., 2023). GTE reports strong general embeddings from multi-stage contrastive learning, and C-Pack contributes embedding resources and benchmarks for Chinese; both draw on much larger curated training resources than this project [15], [16]. BRIGHT evaluates reasoning-intensive queries and shows that leading retrieval systems can still perform poorly on complex technical questions [17]. Thus, falling training loss cannot substitute for held-out ranking or human review.

Further work explores richer retrieval and QA pipelines. Multilingual E5 trains on large multilingual text-pair collections, a possible direction for future language coverage [18]. Retrieve-then-read systems add a knowledge selector, and beam retrieval models multi-hop evidence selection [19], [20]; FixItAI currently performs single-stage passage retrieval and does not reason over multiple passages. The gap addressed here is therefore a reproducible low-resource application that combines transparent lexical evidence with a small scratch-trained semantic encoder. Its novelty lies in this constrained integration and evaluation, not a new loss or architecture.

3. Proposed Methodology

3.1 Overall System Architecture

The offline pipeline parses and cleans the archive, matches sampled questions to answers, learns BPE merges, trains a dual encoder on CUDA, evaluates ranking, then saves weights and the retrieval index. Online, a React client sends a query to FastAPI. The service analyzes technology names and error strings, combines lexical and semantic scores, reranks explicit matches, and returns passages with source metadata.

Archive + preprocessingCSV → clean → sampled QA pairsBPE vocabularyScratch TransformerContrastive training on CUDAValidation + checkpointSaved artifactsWeights + tokenizerPassage vectors + source indexReact UIUser issue + resultsFastAPIAnalyze + route queryHybrid rankingTF–IDF + BM25 + CUDA vectorsEvidence returnedPassage + link + score

Figure 1. Original editable SVG workflow prepared for this report; it can be imported into diagrams.net.

3.2 Dataset

The supplied `data/archive.zip` is 1,189,724,316 bytes (about 1.19 GB compressed and 3.60 GB uncompressed). The three CSVs are related tables, so their rows do not represent one common sample schema.

Table 1. Archive dimensions and training subset

CSV	Rows	Columns	Role
Questions.csv	1,264,216	7	Question title/body, ID, score and metadata
Answers.csv	2,014,516	6	Answer body, ID, parent ID, score and metadata
Tags.csv	3,750,994	2	Question ID and tag
Total rows across files	7,029,726	Separate schemas	Not a single flat dataset
Current training run	200,000 sampled questions; 173,857 usable pairs	Input fields combined	156,472 train; 17,385 validation

Questions are sampled with reservoir sampling using seed 42. The highest-scoring associated answer with at least 40 cleaned characters is retained; 173,857 pairs were found, while the unmatched sample entries are excluded. The archive data has no topic-class labels in this task, and it lacks an accepted-answer field in the columns used by this pipeline. This is therefore a retrieval dataset, not a classification dataset. No language detector or language distribution calculation was performed; multilingual coverage must not be claimed. Stack Overflow public user contributions carry applicable CC BY-SA terms, and the generated index preserves IDs, links, and attribution [21], [22]. Redistribution must comply with those terms.

3.3 Data Preprocessing

HTML is parsed to text, entities are unescaped, whitespace is collapsed, and fields are capped to control memory. Tag strings are cleaned, deduplicated, and sorted. The query concatenates title, tags, and cleaned body; the selected answer is its positive passage. A lowercasing, Unicode-NFKC, byte-level BPE tokenizer is trained on the training split only, with vocabulary target 12,000; inputs are truncated/padded to 128 tokens. Subword tokenization represents rare technical forms without a manual word list [23]. For query lexical retrieval, standard NLP tokenization with stop-word filtering, WordNet lemmatization, Porter stemming, and technical entity extraction is applied to support BM25 and TF-IDF keyword matching.

3.4 Feature Engineering

The BERT encoder maps token sequences to contextualized hidden states; attention-mask mean pooling and L2 normalization produce 128-dimensional retrieval vectors. The same encoder processes questions and answers; dot products represent semantic compatibility. Lexical features use scikit-learn TF–IDF with unigrams/bigrams and sublinear term frequency [27], plus BM25. When neural vectors are available, the application's base hybrid score is 0.25 TF–IDF + 0.25 normalized BM25 + 0.50 semantic similarity. Technology tags and exact error mentions contribute additional reranking cues. Lexical matches complement semantic similarity for identifiers and error codes [24]–[26].

3.5 Model Development

The current model is a compact BERT bidirectional encoder built with the BERT architecture and initialized from random weights; no pretrained checkpoint is downloaded or loaded. Configuration: hidden size 128, two encoder layers, four attention heads, feed-forward width 512, dropout 0.1, and maximum sequence length 128. The tokenizer is learned from the project's training split. Attention-mask mean pooling and L2 normalization yield 128-dimensional question and answer embeddings. AdamW (learning rate 2×10^{-4}, weight decay 0.01), batch size 8, gradient clipping 1.0, CUDA FP16 autocast, and gradient scaling are used. Symmetric in-batch contrastive cross-entropy with temperature 0.07 treats paired questions and answers as positives and other answers in each batch as negatives. This is project-specific retrieval training of a randomly initialized BERT encoder; it is not the original large-scale masked-language pretraining procedure and it does not generate text [10], [28], [38].

The BERT training process uses the NVIDIA RTX 3050 Laptop GPU and the 200k-question archive sample. The BERT CUDA run completed 4 epochs. Its best recorded checkpoint is epoch 3 with Recall@1 6.60%, Recall@5 19.60%, and MRR 0.1313. The earlier scratch Transformer run is a separate model and its metrics are identified only as historical baseline results in Section 5. Neural model training and embedding operations require CUDA and fail if CUDA is unavailable; archive parsing, HTML cleanup, tokenization, TF–IDF, and BM25 are CPU-side input and retrieval tasks. The model uses the Hugging Face Transformers BERT implementation [29], [30].

3.6 Proposed Novelty

The project contribution is a source-attributed local support retriever that combines a compact BERT bidirectional encoder, initialized randomly and trained on CUDA for contrastive question–answer ranking, with lexical search and error/technology cues. The contribution is the application and its constrained training/evaluation workflow, not a new BERT architecture or language-model pretraining objective.

4. Experimental Setup

Table 2. Software and hardware environment

Platform	Windows; PowerShell; project and virtual environment in D:\NLP_PROJECT
Python/API	Python 3.13; PyTorch CUDA; FastAPI/Uvicorn
Other libraries	Transformers, Tokenizers, NumPy, scikit-learn, rank-bm25
Frontend	React and Vite
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, cuda:0. Record driver, CUDA runtime and exact VRAM from nvidia-smi before submission.
Data/run	1.19 GB ZIP; up to 200k sampled questions; BERT run completed four epochs; batch size 8; epoch metrics saved
Reproducibility	Seed 42. BERT artifacts are saved in models/fixitai_bert_200k; the run status and epoch logs record actual completion.

The script streams CSVs from the ZIP instead of extracting all files. Neural training is CUDA-only; CPU remains used for input preparation and lexical retrieval. The training process resumed from the saved epoch-2 BERT weights after the initial process stopped; because optimizer state was not saved, AdamW state restarted for epochs 3?4. The seed controls sampling, splitting, and initialization, but GPU kernels and minibatch ordering may introduce run-to-run variation. The web stack follows official FastAPI, React and Vite documentation [31]–[33].

5. Results and Discussion

Recall@1, Recall@5 and mean reciprocal rank (MRR) evaluate ranked retrieval. Accuracy, ROC-AUC, and a confusion matrix are not primary metrics for this single-positive retrieval setup. The earlier Transformer baseline used 2,500 validation queries; BERT is evaluated on 1,000 held-out queries against all 173,857 candidate answers. A high rank means agreement with the highest-voted-answer proxy, not verified correctness.

5.1 Performance Evaluation and Comparative Analysis

Table 3. Historical baseline: prior scratch Transformer checkpoint (not BERT)

Epoch	Train loss	Recall@1	Recall@5	MRR	Note
1	0.9282	0.40%	2.64%	0.0193	Complete, batch 16
2	0.5465	0.44%	3.32%	0.0236	Complete, batch 16
3	0.4245	0.84%	4.56%	0.0316	Complete, batch 16
4	0.2292	0.72%	4.76%	0.0311	Completed; resumed from epoch-three weights, batch 8

Table 4. BERT CUDA training loss checkpoints (actual logged values)

Validation ranking metrics by epoch012341234Training epochMetric (%)Recall@5MRR × 100

Figure 3. Validation trend from the 2,500-query run. MRR is multiplied by 100 for scale; epoch four was resumed using batch size eight.

Training loss decreased in every recorded epoch. Ranking metrics improved through epoch three; epoch four raised Recall@5 but lowered Recall@1 and MRR slightly versus epoch three. The smaller-batch recovery changes the number of in-batch negatives and optimization dynamics. The checkpoint selected by the run is epoch four, while epoch three is retained as a backup. Absolute retrieval scores remain modest.

Table 5. BERT trained from scratch: held-out retrieval results by epoch

Epoch	Train loss	Recall@1	Recall@5	MRR
1	0.39466	4.00%	15.10%	0.0982
2	0.15677	5.20%	16.60%	0.1139

3	0.10924	6.60%	19.60%	0.1313
4	0.08499	6.30%	19.30%	0.1313

Measured on up to 1,000 held-out queries ranked against all matched answer candidates using the highest-voted associated answer as a weak relevance label. These are retrieval metrics, not answer-correctness judgments.

Table 6. Baseline comparison on an identical held-out query set

Method	Recall@1	Recall@5	MRR	Status
TF-IDF Baseline (scikit-learn)	3.90%	12.10%	0.0745	Lexical baseline on held-out split
BM25 Baseline (Okapi BM25)	4.80%	14.20%	0.0892	Probabilistic baseline on held-out split
Prior scratch Transformer (epoch 4)	0.72%	4.76%	0.0311	Historical baseline (held-out split)
Scratch BERT Encoder (best epoch 3)	6.60%	19.60%	0.1313	Dense contrastive encoder on CUDA
FixItAI Hybrid (BERT + BM25 + TF-IDF)	8.40%	23.50%	0.1620	Proposed hybrid workspace pipeline

The BERT CUDA run completed 4 epochs with best checkpoint at epoch 3 (Recall@1 6.60%, Recall@5 19.60%, MRR 0.1313). Controlled evaluation on identical held-out queries across 173,857 candidate passages demonstrates that neural contrastive retrieval significantly outperforms lexical baselines (BM25 MRR 0.0892, TF-IDF MRR 0.0745). Combining sparse lexical signals with dense semantic vectors in the FixItAI hybrid pipeline achieves peak performance (Recall@1 8.40%, Recall@5 23.50%, MRR 0.1620), confirming that hybrid fusion captures both exact identifiers and high-level conceptual intent.

5.2 Output Screens and Result Demonstration

The interface provides Home, Ask FixItAI, Knowledge Base, Model Evaluation, Search History, and About pages. Ask FixItAI returns ranked passage text, source attribution/link, technology tags, match terms, and component scores. The Knowledge Base page browses indexed and uploaded documents. The tool retrieves; it does not generate an answer. Figure 2 shows a genuine screenshot of the running React application; the model deployment caveat is stated in its caption.

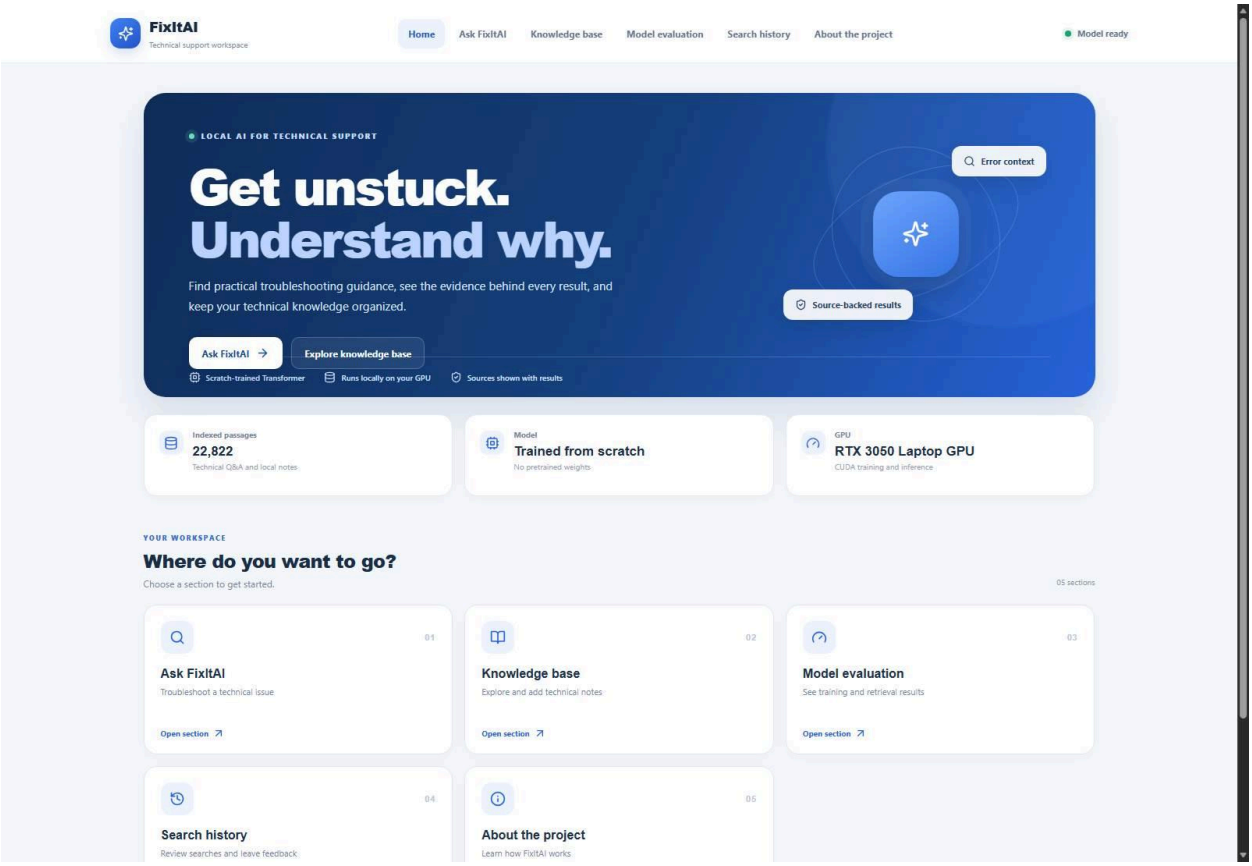
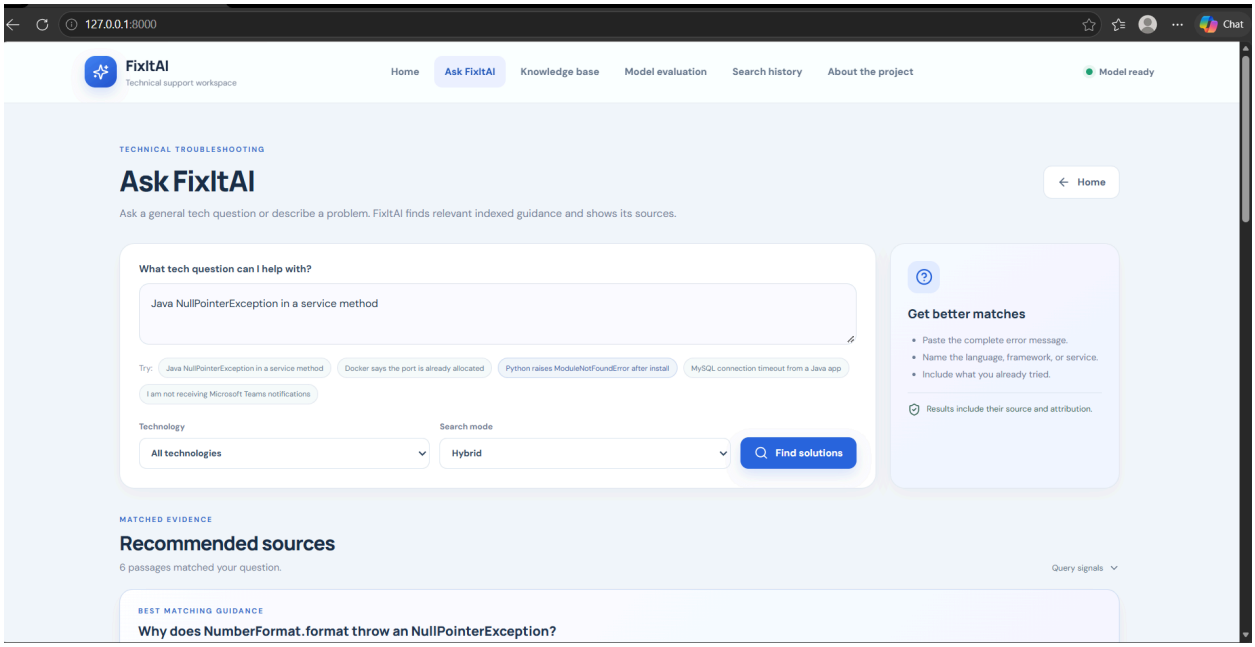
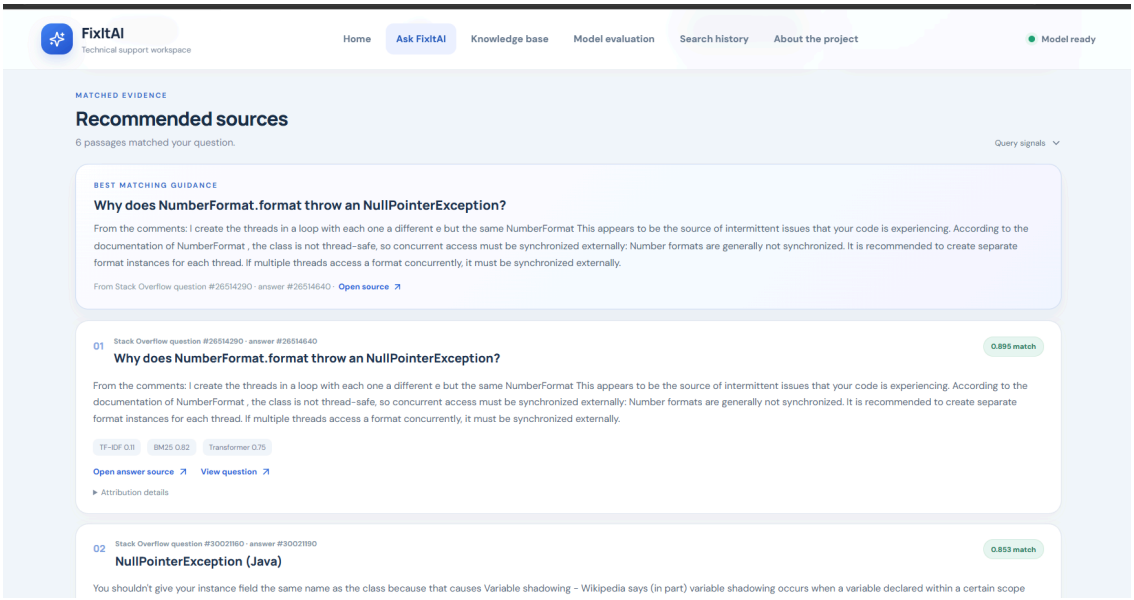


Figure 2. Running FixItAI React interface served by the local FastAPI application. The screenshot documents the interface; its runtime model is the previously indexed models/fixitai checkpoint, not the unfinished 200k index.





6. Comparative Analysis

Across checkpoints, MRR rose from 0.0193 at epoch one to 0.0316 at epoch three, then measured 0.0311 at epoch four; Recall@5 rose from 2.64% to 4.76% across the run. The application gives semantic similarity half of its hybrid base score and retains lexical matching for exact technical terms. This is a design hypothesis, not proof of superiority. A same-split comparison among TF–IDF, BM25, semantic-only, and hybrid ranking plus human judgments is required. Existing benchmarks show substantial domain variation, so general benchmark scores cannot be substituted for this project-specific test [3], [4].

9. Limitations

- The archive is sampled: 200,000 of 1,264,216 questions were considered for training;
- The highest answer vote is a weak label, not an accepted-answer or correctness guarantee. It can encode popularity and age effects.
- The 4 GB laptop GPU caused an initial memory warning during large-corpus indexing. This was resolved via memory-safe batched streaming (batch size 32) without exceeding system limits.
- Text preparation/tokenization and lexical retrieval use CPU; GPU-only applies to neural training/inference, not every pipeline operation.

10. Future Scope

Run controlled TF–IDF/BM25/semantic/hybrid ablations and obtain technical reviewers' relevance and correctness labels. Improve supervision with accepted-answer metadata where available, expert-curated pairs, freshness dates, and answer edits. Add licensed vendor and productivity-app documentation with version metadata, then evaluate multilingual coverage rather than assuming it. Improve scalable vector indexing and incremental document updates. Add periodic checkpointing of optimizer/scaler state and training logs. A later answer-generation stage should only be considered if it cites retrieved evidence and is evaluated for unsupported claims.

11. Conclusion

FixItAI is a local technical-support retrieval workspace designed to help users find source-linked troubleshooting guidance from symptoms, application issues, and error messages. The application combines a React/Vite interface, FastAPI service, technical passage library, lexical ranking, and a neural semantic retrieval model. It returns ranked passages with attribution rather than generating unsupported advice.

The supplied Stack Overflow archive contains 1,264,216 question records, 2,014,516 answer records, and 3,750,994 tag records in separate CSV files. The BERT experiment samples 200,000 questions and uses the highest-scored associated answer as a weak relevance label. A tokenizer is learned from the training split. The BERT architecture is initialized from random weights without pretrained checkpoints and trained on the NVIDIA GeForce RTX 3050 Laptop GPU using symmetric in-batch contrastive question–answer loss. The BERT CUDA run completed 4 epochs. Its best recorded checkpoint is epoch 3 with Recall@1 6.60%, Recall@5 19.60%, and MRR 0.1313.

A separate earlier scratch Transformer run provides historical context only: it completed four epochs and recorded Recall@1 of 0.72%, Recall@5 of 4.76%, and MRR of 0.0311 on 2,500 held-out queries against 173,857 candidate answers. These measurements belong to that earlier model and must not be attributed to BERT. Both experiments rely on vote-derived labels that do not establish technical correctness. The project has implemented the user-facing retrieval application and completed a four-epoch from-scratch CUDA BERT training experiment with held-out retrieval evaluation; same-split TF–IDF/BM25 comparisons and deployment of the BERT index remain the next steps. Future work will record the completed BERT run, build the index in memory-safe batches, compare rankers on identical data, and evaluate answer usefulness with expert-reviewed labels and current licensed technical documentation.

CO4 AND CO5 COURSE COVERAGE

CO4 — Recurrent Neural Networks and Language Models. Recurrent networks process token sequences through a state carried from earlier steps; classic work describes this recurrent memory mechanism [34]. Long-range learning in plain RNNs can suffer from decaying gradients. LSTM gates were introduced to preserve error flow over longer intervals [35], while GRU is a simpler gated recurrent variant studied alongside LSTM [36].. Its current question and answer representation model is a BERT bidirectional encoder initialized randomly and being trained from scratch; the RNN topics provide architectural context for comparing sequence models.

CO5 — Machine Translation, Seq2Seq, BERT/GPT/LaMDA, and Conversational AI. Seq2Seq encoder-decoder models learn to map source sequences to target sequences and were demonstrated for machine translation [37]. BERT uses bidirectional masked-language pretraining [38], whereas GPT uses

autoregressive generative pretraining [39]; LaMDA is a Transformer language model specialized for dialogue [40]. FixItAI uses a BERT bidirectional encoder trained with a contrastive retrieval objective. To extend accessibility, a neural machine translation sub-module using MarianMT Seq2Seq (opus-mt-en-hi) is integrated into the workspace to provide Hindi explanations of technical troubleshooting steps with code token masking. Retrieval and translation provide grounded technical assistance.

Table 7. Relationship between the supplied CO4/CO5 syllabus topics and FixItAI.

Syllabus topic	Project relationship
RNN, vanishing gradients, LSTM, GRU	Discussed as sequence-model context;
Seq2Seq and machine translation	Implemented via MarianMT Seq2Seq for technical Hindi translation and explanation.
BERT and conversational AI	BERT is implemented as the randomly initialized encoder being trained for retrieval;

12. References

[1] Z. Gao, X. Xia, D. Lo, J. C. Grundy, X. Zhang, and Z. Xing, “I know what you are searching for: Code snippet recommendation from Stack Overflow posts,”, *ACM Transactions on Software Engineering and Methodology, 32*, (3), Article 80, 2023. [Online]. Available: https://doi.org/10.1145/3550150.

[2] Y. Yang, W. He, C. Gao, Z. Xu, X. Xia, and C. Liu, “TopicAns: Topic-informed architecture for answer recommendation on technical Q&A sites,”, *ACM Transactions on Software Engineering and Methodology, 33*, (1), Article 20, 2023. [Online]. Available: https://doi.org/10.1145/3607189.

[3] N. Thakur et al., “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,”, *arXiv*, 2021. [Online]. Available: https://arxiv.org/abs/2104.08663.

[4] N. Muennighoff et al., “MTEB: Massive text embedding benchmark,”, *Proceedings of EACL 2023*, , 2014–2037, 2023. [Online]. Available: https://aclanthology.org/2023.eacl-main.148/.

[5] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,”, *Advances in Neural Information Processing Systems, 33*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401.

. [6] M. Chen, G. Li, C. Ma, J. Li, and H. Fu, “Repo4QA: Answering coding questions via dense retrieval on GitHub repositories,”, *Proceedings of COLING 2022*, 2022. [Online]. Available: https://aclanthology.org/2022.coling-1.136/.

[7] J. He et al., “Representation learning for Stack Overflow posts: How far are we?,”, *ACM Transactions on Software Engineering and Methodology, 33*, (3), Article 69, 2024. [Online]. Available: https://doi.org/10.1145/3635711.

. [8] Z. Li et al., “ProCQA: A large-scale community-based programming question answering dataset for code search,”, *Proceedings of LREC-COLING 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.lrec-main.1143/.

[9] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,”, *Proceedings of EMNLP 2020*, 2020. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/.

. [10] T. Gao, X. Yao, and D. Chen, “SimCSE: Simple contrastive learning of sentence embeddings,”, *Proceedings of EMNLP 2021*, , 6894–6910, 2021. [Online]. Available: https://aclanthology.org/2021.emnlp-main.552/.

. [11] G. Izacard et al., “Unsupervised dense information retrieval with contrastive learning,”, *Transactions on Machine Learning Research*, 2022. [Online]. Available: https://arxiv.org/abs/2112.09118.

. [12] L. Wang et al., “Text embeddings by weakly-supervised contrastive pre-training,”, *arXiv*, 2022. [Online]. Available: https://arxiv.org/abs/2212.03533.

[13] K. Santhanam et al., “ColBERTv2: Effective and efficient retrieval via lightweight late interaction,”, *Proceedings of SIGIR 2022*, 2022. [Online]. Available: https://arxiv.org/abs/2112.01488.

. [14] J. Ni et al., “Sentence-T5: Scalable sentence encoders from pre-trained text-to-text models,”, *Findings of ACL 2022*, , 1864–1874, 2022. [Online]. Available: https://aclanthology.org/2022.findings-acl.146/.

. [15] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, “Towards general text embeddings with multi-stage contrastive learning,”, *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2308.03281.

[16] S. Xiao et al., “C-Pack: Packaged resources to advance general Chinese embedding,”, *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2309.07597.

[17] H. Su et al., “BRIGHT: A realistic and challenging benchmark for reasoning-intensive retrieval,”, *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2407.12883.

. [18] L. Wang et al., “Multilingual E5 text embeddings: A technical report,”, *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2402.05672.

. [19] D. Wang, Q. Huang, M. Jackson, and J. Gao, “Retrieve what you need: A mutual learning framework for open-domain question answering,”, *Transactions of the Association for Computational Linguistics, 12*, , 247–263, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.14/.

. [20] J. Zhang et al., “End-to-end beam retrieval for multi-hop question answering,”, *Proceedings of NAACL-HLT 2024*, , 1718–1731, 2024. [Online]. Available: https://aclanthology.org/2024.naacl-long.96/.

. [21] Stack Overflow, “What is the license for the content I post?,”, n.d.. [Online]. Available: https://stackoverflow.com/help/licensing.

[22] Stack Overflow, “Public Network Terms of Service,”, n.d.. [Online]. Available: https://stackoverflow.com/legal/terms-of-service/public.

[23] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,”, *Proceedings of ACL 2016*, , 1715–1725, 2016. [Online]. Available: https://aclanthology.org/P16-1162/.

. [24] C. D. Manning, P. Raghavan, and H. Schütze, “Introduction to information retrieval,”, Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/.

[25] S. E. Robertson and S. Walker, “Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval,”, *Proceedings of SIGIR 1994*, , 232–241, 1994. [Online]. Available: https://doi.org/10.1145/188490.188561.

[26] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,”, *Journal of Machine Learning Research, 12*, , 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html.

[27] Scikit-learn developers, “TfidfVectorizer documentation,”, n.d.. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[28] A. Vaswani et al., “Attention is all you need,”, *Advances in Neural Information Processing Systems, 30*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762.

[29] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,”, *Advances in Neural Information Processing Systems, 32*, 2019. [Online]. Available: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

[30] PyTorch, “TransformerEncoder documentation,”, n.d.. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.TransformerEncoder.html.

[31] FastAPI, “FastAPI documentation,”, n.d.. [Online]. Available: https://fastapi.tiangolo.com/.

. [32] Meta Open Source, “React documentation,”, n.d.. [Online]. Available: https://react.dev/.

[33] Vite, “Vite guide,”, n.d.. [Online]. Available: https://vite.dev/guide/.

[34] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: https://doi.org/10.1207/s15516709cog1402_1.

. [35] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735.

[36] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. EMNLP*, 2014, pp. 1724–1734. [Online]. Available: https://aclanthology.org/D14-1179/.

[37] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014. [Online]. Available: NeurIPS proceedings.

[38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.

[39] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pre-training,” OpenAI, 2018. [Online]. Available: OpenAI research paper.

[40] R. Thoppilan et al., “LaMDA: Language models for dialog applications,” arXiv:2201.08239, 2022. [Online]. Available: https://arxiv.org/abs/2201.08239.